RoboCup RoboCup Junior **Brazil Rescue Maze**

FRAN ROBOTS

SESI



Fernanda Lima
3D Designer; captain
Designs and assembles the robot; coordinates team actions.



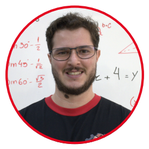
José Faccirolli
Programmer; vice-captain
Programs the robot's navegation and image processing.



Lívia Comparini
Designer
Documents all team activities; assembles the robot.



Valentina Franco
Electronics Designer
Designs and builds the robot's electrical circuits.



Breno Malta
Instructor
Guides the team and manages administrative tasks.

1. Introduction

We present the results of a project started in **August 2023**. Our previous experience in the **RoboCupJunior Rescue Line** (**OBR**) was crucial. This poster highlights the team's progress over the period of almost one year. Through it, we aim to share the **knowledge** and **new skills** we have acquired so far.

1.1. History

The Fran Robots team has a **successful history** in the RoboCupJunior **Rescue Maze !**

- 1º RCJ Rescue Maze - Nacional (2023)
- 1º RCJ Rescue Maze - Regional (2024)
- 2º RCJ Rescue Maze - Nacional (2024)
- 1º RCJ Rescue Maze - Regional (2025)
- 2º RCJ Rescue Maze - Nacional (2025)



National stage in Vitória, ES

Our GitHub! GitHub

A little bit more about us!

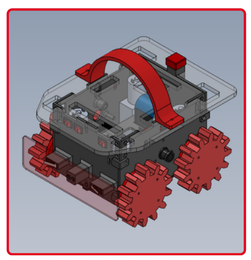
@fran_robots
sesifranrobots@gmail.com
School SESI 109 (Franca-SP)

2. Robot Design and Electronics

2.1. General Structure

Our robot features **modular architecture** with **3D-printed ABS** parts for better strength and durability. With that structure, it is possible to have all necessary pieces in only a **few** parts. Also, we have a roof made out of **transparent acrylic** for better inside visibility.

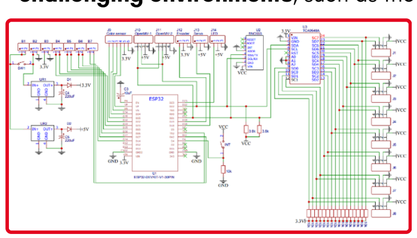
A **tactile bumper** equipped with microswitches assists with alignment and obstacle detection. The structure's **modular design** provides easy access to components, allowing **quick maintenance** during the competition if needed.



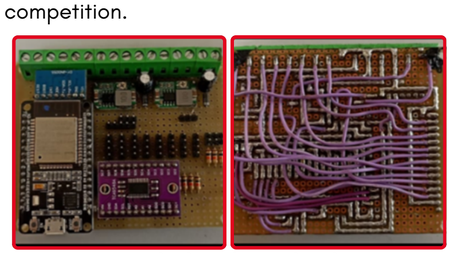
Robot's 3D Model

2.2. Electronics

We use **EasyEDA** software to **create schematics** and document the robot's circuits. **Our mother board** is **hand-soldered** onto perfboards, which simplifies maintenance. Although **we have tested PCB fabrication**, we opted for perfboards because errors would otherwise require producing a new board. This **manual method allows for quick adjustments** and ensures the robot's effectiveness **in challenging environments**, such as the competition.



Motherboard Schematic



Motherboard Completed

1 **ESP32 manages movements** and **decisions, processing sensor data** for distance, color, orientation, and victim classification.





2 **BNO055** acts both as **a gyroscope** and to **track the robot's tilt**, ensuring accurate position and orientation measurements.

3 Two DC-DC **Step Down** Mini 360 3a **voltage regulators** for **regulating 12V** to 3.3V and 5V.





4 **PCA9548A I2C multiplexer** allows the use of several identical sensors on the same bus, activating one channel at a time and **avoiding address conflicts**.

5 **GY-VL53L0XV2** ToF distance sensors for **detecting** labyrinth **walls**.





6 **OpenMV H7 Plus** camera for image processing in **MicroPython**, using the **OpenMV IDE.**

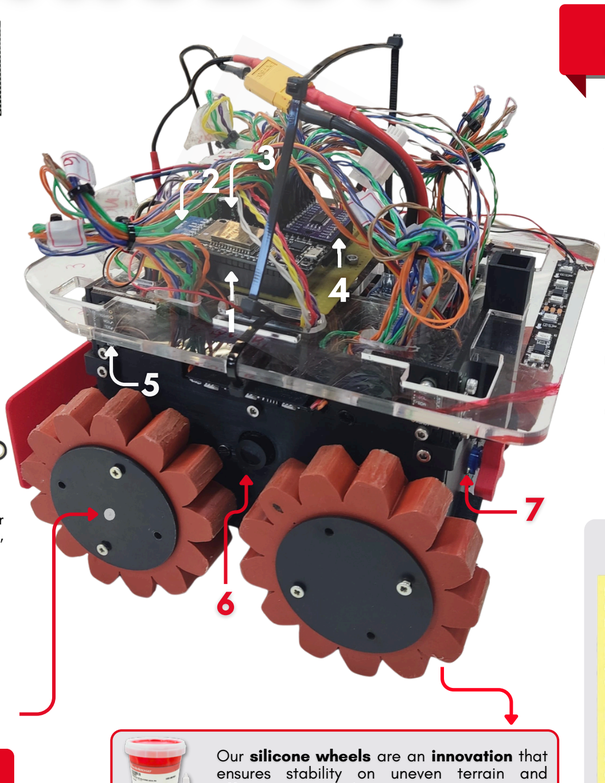
7 9g **SG90 Servo Motor to fire** rescue **kits on** both sides.





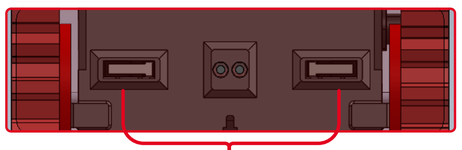
50RPM motors with **reduction box** and **encoder** for moving the robot.

Estimated cost of the robot: C$ 1,354.68

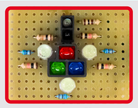




Our **silicone wheels** are an **innovation** that ensures stability on uneven terrain and efficient ramp climbing.



Two **microswitches** positioned on the front **bumper** allow the robot to **detect collisions** with potential **obstacles**. Contact on just one side of the bumper enables the robot to identify the direction in which to steer **to avoid** the obstacle.



- C9L3 sensor (infrared);
- LDRs with RGB filters;
- High-brightness LEDs.

The **reflectance plate** is responsible for **identifying** the **colors of the floor** surface. White light emitted by **LEDs** reflects off the floor, passes through **red**, **green**, or **blue** (RGB) filters, and triggers specific actions based on the result.

Additionally, the **infrared sensor** analyzes the surface based on the intensity of the **reflected** light.

2.3. Kit Launching

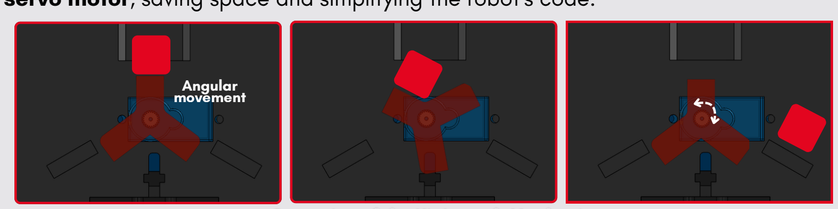
During a previous competition, we encountered issues with the rescue kits, which would **bounce** because they were **too light**. To solve this, we placed three to four small **solder** balls **inside** the kit, increasing its weight and **stabilizing the center of mass** during launch. This solution **prevents the kits** from bouncing and **falling outside the 15 cm radius line.**



Before and After!

2.4. Kit Launching Mechanism

Our rescue kit deployment mechanism operates like an **inverted "Y"** system that **moves smoothly to the right or left**, releasing kits in both directions **using a single servo motor**, saving space and simplifying the robot's code.

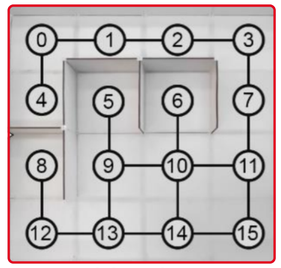


Operation of the Kit Launch Mechanism

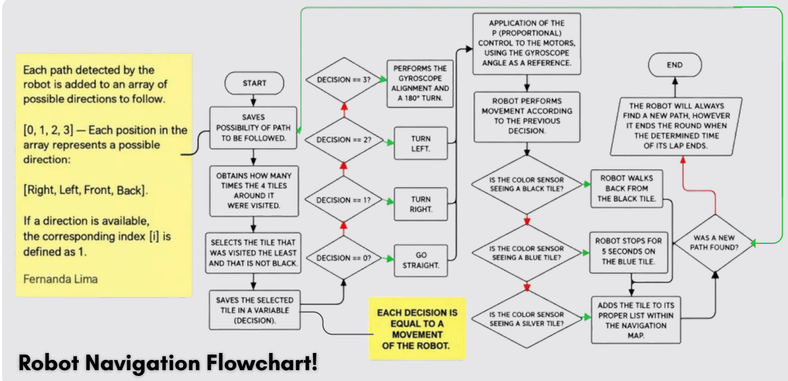
3. Software

3.1. Maze Navigation and Mapping

The robot employs a **graph-based system**, programmed in **C++**, to navigate the maze by treating it as a grid of **points** (vertices) and **connections** (edges). It maps the environment in real time, identifying walls, paths, and colored tiles via its sensors. Building this map enables the robot to find the fastest routes and **prioritize unexplored areas**, ensuring precise movement through the use of **PD** controls. A **custom library** facilitates navigation and mapping, while public libraries optimize the use of sensors and actuators. This system ensures rapid and efficient decision-making within the maze.



Graph-based System



Robot Navigation Flowchart!

3.2. Image Recognition

Our robot uses two **OpenMV H7 Plus cameras** for victim detection. For letter-shaped victims, we employ a **machine learning model** based on **MobileNetV2**, trained on **Edge Impulse**. Images are resized, converted to grayscale, and classified directly on the camera.

For colored victims, we use filtering in the LAB color space and validate the size and shape of the detection. All results are sent to the **ESP32 via I2C**. The system is fast and reliable, achieving over 95% confidence in most cases.

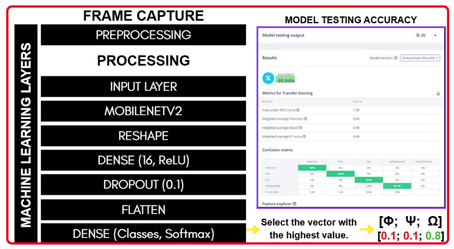


Image Recognition Processes

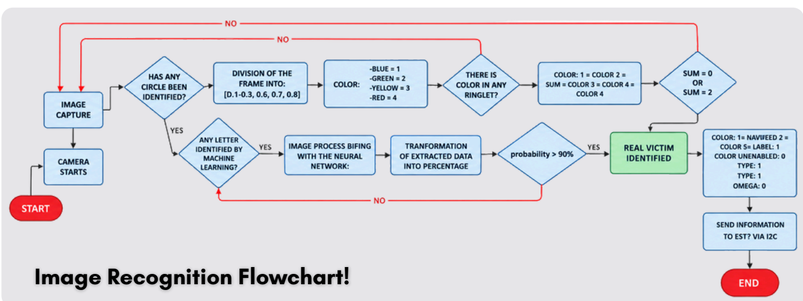


Image Recognition Flowchart!